Deployment Manual: Data Weaver-Distributed Agentic Data Pipeline

1. Overview

The **Distributed Agentic Data Pipeline** is a multi-node AI system that automates dataset integration using a **two-agent architecture**:

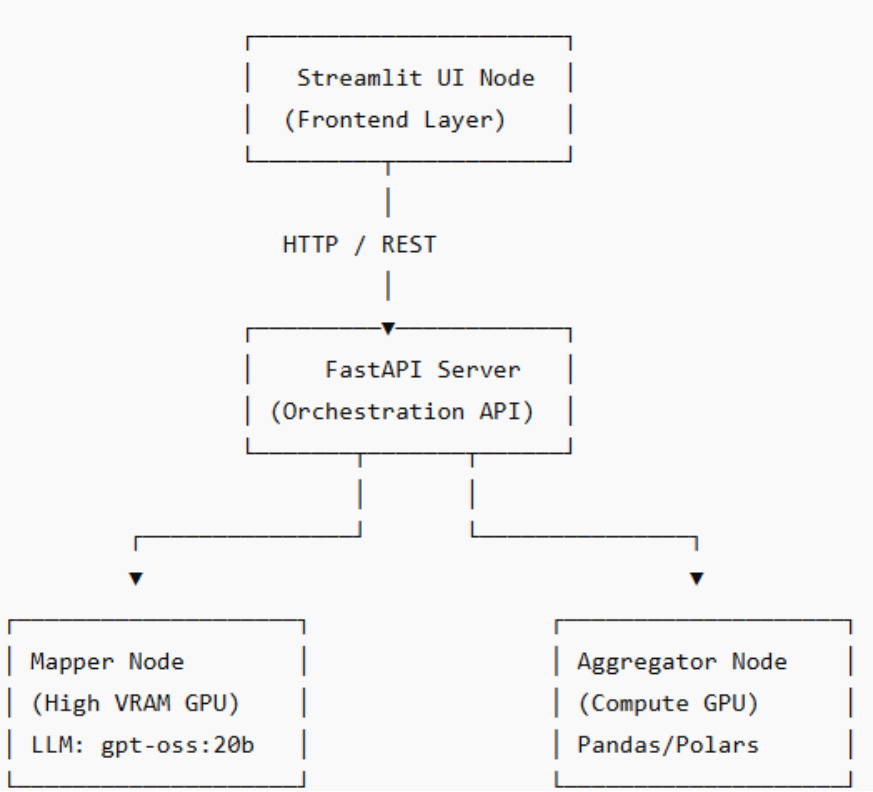
- **Mapper Agent (Node 1)** → Schema inference & ERD generation
- **Aggregator Agent (Node 2)** → Data transformation, joins, validation

The system is orchestrated using:

- **LangGraph** (agent workflow)
- **FastAPI** (service layer)
- **Streamlit** (UI dashboard)
- **Tailscale** (secure node-to-node networking)

2. Target Deployment Architecture

Production Topology



3. Prerequisites

Hardware Requirements

Component	Minimum Requirement
Mapper Node	24–36 GB VRAM GPU
Aggregator Node	12–16 GB VRAM GPU
API Server	8 GB RAM
UI Server	4 GB RAM

Software Requirements

- Python **3.10+**
- Docker (recommended for production)
- Git
- Tailscale
- CUDA Toolkit (if GPU-enabled)

4. Repository Setup

Step 1: Clone Repository

```
git clone https://github.com/htmw/S2026A-Team2.git
cd S2026A-Team2
```

Step 2: Environment Setup

Create virtual environments per service:

```
python -m venv venv
source venv/bin/activate   # Linux/macOS
venv\Scripts\activate      # Windows
```

Install dependencies:

```
pip install -r requirements.txt
```

5. Environment Configuration

Create a .env file in the root:

```
# API
API_HOST=0.0.0.0
API_PORT=8000

# Agent Nodes
MAPPER_NODE_URL=http://<mapper-node-ip>:8001
AGGREGATOR_NODE_URL=http://<aggregator-node-ip>:8002

# Model Configuration
```

MODEL_NAME=gpt-oss:20b

# Data Paths
DATA_INPUT_PATH=/data/input
DATA_OUTPUT_PATH=/data/output

# Tailscale Network
TAILSCALE_AUTH_KEY=<your-auth-key>

6. Network Setup (Tailscale)

Step 1: Install Tailscale

curl -fsSL https://tailscale.com/install.sh | sh

Step 2: Authenticate

tailscale up --authkey <your-auth-key>

Step 3: Verify Connectivity

tailscale status
ping <peer-node-name>

Ensure all nodes:

- Can reach each other via private Tailscale IPs
- Use those IPs in .env

7. Model Deployment

Step 1: Install Model Runtime

Depending on your implementation (e.g., Ollama or local runner):

ollama pull gpt-oss:20b

Step 2: Run Model Service

ollama serve

Ensure:

- Model is accessible via API
- Ports are open internally (not public)

8. Service Deployment

8.1 Mapper Node Deployment

cd services/mapper
uvicorn main:app --host 0.0.0.0 --port 8001

Responsibilities:

- Schema inference

- ERD generation

8.2 Aggregator Node Deployment

```
cd services/aggregator
uvicorn main:app --host 0.0.0.0 --port 8002
```

Responsibilities:

- Data joins
- Schema validation
- Invariant checks

8.3 FastAPI Orchestrator

```
cd services/api
uvicorn main:app --host 0.0.0.0 --port 8000
```

Responsibilities:

- Workflow orchestration
- Routing requests between agents

8.4 Streamlit UI Deployment

```
cd ui
streamlit run app.py --server.port 8501
```

9. Docker-Based Deployment (Recommended)

Build Images

```
docker build -t agentic-api ./services/api
docker build -t mapper-node ./services/mapper
docker build -t aggregator-node ./services/aggregator
docker build -t streamlit-ui ./ui
```

Docker Compose Example

```
version: '3.8'

services:
  api:
    image: agentic-api
    ports:
      - "8000:8000"
    env_file:
      - .env

  mapper:
    image: mapper-node
    ports:
```

```
    - "8001:8001"

  aggregator:
    image: aggregator-node
    ports:
      - "8002:8002"

  ui:
    image: streamlit-ui
    ports:
      - "8501:8501"
```

Run:

```
docker-compose up -d
```

10. Data Pipeline Execution

Trigger Pipeline

POST /run-pipeline

Payload:

```
{
  "input_files": ["customers.csv", "orders.csv"]
}
```

Expected Outputs

- Cleaned dataset
- ERD visualization
- Data dictionary
- Validation logs

11. Monitoring & Logging

Logs

- API logs → /logs/api.log
- Mapper logs → /logs/mapper.log
- Aggregator logs → /logs/aggregator.log

Health Check Endpoints

```
GET /health
GET /mapper/status
GET /aggregator/status
```

12. Security Considerations

- Use **Tailscale private network** (no public exposure)
- Enable **API authentication (JWT recommended)**
- Restrict ports using firewall rules
- Store secrets in **vault or environment variables**

13. Scaling Strategy

Horizontal Scaling

- Add more Aggregator nodes for parallel processing
- Load balance via API gateway

Vertical Scaling

- Increase GPU VRAM for Mapper node (improves reasoning quality)

14. Failure Handling

Failure Type	Mitigation
Node failure	Retry via orchestrator
Model timeout	Fallback to cached schema
Data explosion	Enforced row-count invariant

15. Rollback Strategy

- Maintain versioned Docker images
- Use tagged releases:

```
docker pull agentic-api:v1.2
```

- Rollback via:

```
docker-compose down
docker-compose up -d --build
```

16. Post-Deployment Validation

Checklist:

- All services reachable
- Tailscale network stable
- Model responding
- Pipeline executes end-to-end
- UI reflects results correctly

17. Known Limitations

- High VRAM dependency for Mapper agent
- Latency increases with dataset size
- Requires stable inter-node networking

18. Conclusion

This deployment setup enables a **production-grade distributed AI pipeline** with:

- Deterministic validation (not just LLM guessing)
- Modular scaling across nodes
- Clear separation of reasoning vs execution